

BinarySearch – Masterfile (Deutsch + فارسی)

این فایل تمام نسخه‌های مهم BinarySearch را شامل می‌شود: نسخه استاندارد (Iterativ)، نسخه بازگشتی (Rekursiv)، نسخه آبجکت، نسخه جنریک با `<Function<T`، خطایابی و تمرین‌های شبیه امتحان GA2.

1. Standard BinarySearch (Iterativ)

```
funktion binarySearch(a: Array von Integer, x: Integer) : Integer
  links  = 0
  rechts = a.length - 1

  solange links <= rechts
    mitte = (links + rechts) / 2

    wenn a[mitte] = x dann
      return mitte
    sonst wenn a[mitte] < x dann
      links = mitte + 1
    sonst
      rechts = mitte - 1
    ende wenn
  ende solange

  return -1
end funktion
```

این نسخه استاندارد و پرتکرار در امتحان است. شرط مهم: `links <= rechts` ، و در صورت عدم پیدا شدن مقدار، 1- برگردانده می‌شود.

2. Rekursiver BinarySearch

```
funktion binarySearchRekursiv(  
    a: Array von Integer,  
    x: Integer,  
    links: Integer,  
    rechts: Integer) : Integer  
  
    wenn links > rechts dann  
        return -1  
    ende wenn  
  
    mitte = (links + rechts) / 2  
  
    wenn a[mitte] = x dann  
        return mitte  
    sonst wenn a[mitte] < x dann  
        return binarySearchRekursiv(a, x, mitte + 1, rechts)  
    sonst  
        return binarySearchRekursiv(a, x, links, mitte - 1)  
    ende wenn  
end funktion
```

در نسخه بازگشتی، به جای حلقه از فراخوانی خود تابع استفاده می‌شود. شرط پایان: `links > rechts`.
مزیت: خوانایی؛ عیب: مصرف بیشتر Stack.

3. BinarySearch für Objekte – جستجو روی آبجکت‌ها

```
funktion sucheKunde(  
    kunden: Array von Kunde,  
    nummer: Integer) : Integer  
  
    links = 0  
    rechts = kunden.length - 1  
  
    solange links <= rechts  
        mitte = (links + rechts) / 2  
  
        wenn kunden[mitte].getNummer()  
            = nummer dann  
            return mitte  
  
        sonst wenn kunden[mitte].getNummer()  
            < nummer dann  
            links = mitte + 1  
  
        sonst  
            rechts = mitte - 1  
        ende wenn  
    ende solange  
  
    return -1  
end funktion
```

در اینجا آرایه از نوع **Kunde** است و مقایسه با متد **getNummer()** انجام می‌شود. الگو دقیقاً مثل نسخه استاندارد است.

4. Generischer BinarySearch mit Function<T>

```
funktion binarySearch(  
    liste: List<T>,  
    x: T,  
    f: Function<T> : Integer  
  
    links = 0  
    rechts = liste.size() - 1  
  
    solange links <= rechts  
        mitte = (links + rechts) / 2  
  
        cmp = f(liste.get(mitte), x)  
  
        wenn cmp = 0 dann  
            return mitte  
        sonst wenn cmp < 0 dann  
            // Element in der Mitte ist kleiner als x  
            links = mitte + 1  
        sonst  
            // Element in der Mitte ist größer als x  
            rechts = mitte - 1  
        ende wenn  
    ende solange  
  
    return -1  
end funktion
```

در نسخه جنریک، تابع f یک مقایسه کننده است: اگر $cmp < 0$ باشد یعنی عنصر وسط x است و باید سمت راست را جستجو کنیم. اگر $cmp > 0$ باشد یعنی عنصر وسط x است و باید سمت چپ را جستجو کنیم.

5. Fehleranalyse – خطایابی BinarySearch

5.1 Fehlerhafte Version

```
funktion sucheFehler(a: Array von Integer, x: Integer) : Integer
    links  = 0
    rechts = a.length           // Fehler: sollte a.length - 1 sein

    solange links < rechts      // Fehler: sollte <= sein
        mitte = (links + rechts) / 2

        wenn a[mitte] = x dann
            return mitte
        sonst wenn a[mitte] > x dann
            links = mitte + 1 // Fehler: sollte rechts ändern
        sonst
            rechts = mitte - 1
        ende wenn
    ende solange

    return 0                    // Fehler: 0 statt -1
end funktion
```

5.2 Korrigierte Version

```
funktion suche(a: Array von Integer, x: Integer) : Integer
    links  = 0
    rechts = a.length - 1

    solange links <= rechts
        mitte = (links + rechts) / 2

        wenn a[mitte] = x dann
            return mitte
        sonst wenn a[mitte] < x dann
            links = mitte + 1
        sonst
            rechts = mitte - 1
        ende wenn
    ende solange

    return -1
end funktion
```

اشتباه‌های کلاسیک: استفاده از `a.length` به‌جای `a.length - 1` ، استفاده از شرط `links <`
`rechts` به‌جای `links <= rechts` ، تغییر دادن متغیر اشتباه در شاخه `a[mitte] > x` ، و
برگرداندن مقدار نادرست در حالت «پیدا نشد».

6. Übungsaufgaben (GA2-Stil) – تمرین های شبیه امتحان

Übung 1 – Iterativer Ablauf

gegeben:

`a = [3, 7, 9, 12, 18, 21]`

`x = 12`

Aufgabe:

Geben Sie die Werte von `links`, `rechts` und `mitte` in jedem Schleifendurchlauf an.

Übung 2 – BinarySearch für Objekte

Kunden sind nach Kundennummer aufsteigend sortiert.

Gesucht ist der Kunde mit der Nummer 45021.

Aufgabe:

Schreiben Sie eine Funktion

`sucheKunde(kunden: Array von Kunde, nummer: Integer) : Integer`,
die mit BinarySearch arbeitet.

Übung 3 – Generischer BinarySearch

Es ist eine `List<Artikel>` nach Preis sortiert.

Eine Funktion `<Artikel> f(a, b)` liefert:

`< 0` wenn `a` billiger als `b` ist,

`= 0` wenn Preis gleich.

`> 0` wenn `a` teurer als `b` ist.

Aufgabe:

Implementieren Sie `binarySearch(liste, artikel, f)`.

Übung 4 – Fehlerkorrektur

Im folgenden Algorithmus sind Fehler:

```
funktion suche(a: Array von Integer, x: Integer) : Integer
    links  = 0
    rechts = a.length
    solange links < rechts
        mitte = (links + rechts) / 2
        wenn a[mitte] = x dann
            return mitte
        sonst wenn a[mitte] > x dann
            links = mitte + 1
        sonst
            rechts = mitte + 1
        ende wenn
    ende solange
    return 0
end funktion
```

Aufgabe:

Finden und korrigieren Sie alle Fehler.

Übung 5 – Theoriefragen

1. Warum benötigt BinarySearch ein sortiertes Array?
2. Wie ist die Laufzeit von BinarySearch im Worst Case?
3. Nennen Sie einen Vorteil von BinarySearch gegenüber LinearSearch.
4. Nennen Sie einen Vorteil von LinearSearch gegenüber BinarySearch.
5. Was passiert, wenn links < rechts statt links <= rechts verwendet wird?